



Πανεπιστήμιο Πειραιώς
Σχολή Τεχνολογιών Πληροφορικής και Τηλεπικοινωνιών
Τμήμα Ψηφιακών Συστημάτων

Επίπεδο: Μεταπτυχιακό Πρόγραμμα Σπουδών

Ασφάλεια Δικτύων

Case 3

Επιβλέπον Καθηγητής: Καθ. Χρήστος Ξενάκης

Καλαϊτζίδης Ιωάννης

ioannis_kalaitzidis@ssl-unipi.gr

Πειραιάς
04/11/2025

Περίληψη

Υλοποίηση επίθεσης man-in-the-middle με έναν επιτιθέμενο (Kali Linux) και ένα θύμα (Ubuntu). Ο επιτιθέμενος τοποθετείται ανάμεσα στο θύμα και το gateway μέσω arpspoofing για να παρακολουθεί τη σύνδεση. Χρήση του packet sniffer από το Case 2 για την καταγραφή της κίνησης σε .pcap αρχείο. Ανάλυση της καταγραφής με Wireshark για εξαγωγή συμπερασμάτων.

Περιεχόμενα

Εισαγωγή	1
Πρακτικό κομμάτι εργασίας	2
Βιβλιογραφία	13

Εισαγωγή

Για τη συγκεκριμένη άσκηση, που αποτελεί συνέχεια της προηγούμενης, θα χρειαστεί να χρησιμοποιήσουμε ένα ακόμη λειτουργικό σύστημα στο VMware. Θα επιλέξουμε το Ubuntu 24.04.3 ως θύμα και το Kali Linux ως επιτιθέμενο. Και στα δύο λειτουργικά συστήματα πρέπει να έχουμε τον ίδιο τύπο διεπαφής δικτύου Bridge ώστε να βρίσκονται στο ίδιο δίκτυο και να μπορούν να επικοινωνούν μεταξύ τους. Επιπλέον ο network adapter NAT παρέχει τις IP στις εικονικές μηχανές μέσω του εικονικού δικτύου του host, ενώ στο Bridge mode οι IP δίνονται απευθείας από το router. Αν και στην αρχή είχαμε ξεκινήσει με το NAT, παρατήρησαμε ότι διακοπτόταν η σύνδεση του θύματος με το gateway/εικονικό router αμέσως μετά τη χρήση του arpspoof και για αυτό το λόγο ξανακάναμε την άσκηση με Bridge.

Πρακτικό κομμάτι εργασίας

Εισαγωγή:

Αρχικά θα εντοπίσουμε τις διευθύνσεις IP και στα δύο συστήματα και στη συνέχεια, με το εργαλείο nmap, θα επιβεβαιώσουμε ότι η IP του Ubuntu αντιστοιχεί στο θύμα. Έπειτα, με το εργαλείο arpspoof, θα τοποθετηθούμε ως man-in-the-middle στην επικοινωνία μεταξύ Ubuntu και router: το arpspoof θα στέλνει ψευδείς ARP απαντήσεις (ARP replies) ώστε το θύμα να «βλέπει» τη MAC Address του gateway ως τη MAC Address του Kali Linux και το gateway να «βλέπει» τη MAC Address του θύματος ως τη MAC Address του Kali Linux. Σκοπεύουμε να μην διακόψουμε τη σύνδεση μεταξύ θύματος και router, αλλά απλώς να παρακολουθήσουμε την κίνηση που ανταλλάσσεται ανάμεσά τους. Αφού ολοκληρώσουμε τη διαδικασία με το arpspoof, θα ενεργοποιήσουμε το promiscuous mode και θα τρέξουμε τον sniffer από το Case 2 ώστε να καταγράψει τα πακέτα σε αρχείο .pcap. Μετά το πέρας της καταγραφής, θα ανοίξουμε το .pcap αρχείο με το εργαλείο Wireshark για ανάλυση και εξαγωγή συμπερασμάτων.

Εκκίνηση διαδικασιών:

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)~$ ifconfig
br-480f97c7b15a: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.18.0.1 netmask 255.255.0.0 broadcast 172.18.255.255
    ether 02:42:b8:64:a7:dc txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:ba:30:70:88 txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 6 overruns 0 carrier 0 collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.109 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::2a02:1388:4001:265d:94f8:efb5:9279:c05c prefixlen 64 scopeid 0<global>
    inet6 fe80::30ed:f8e6:63b6:348c prefixlen 64 scopeid 0<link>
    ether 00:0c:29:fe:e8:b5 txqueuelen 1000 (Ethernet)
    RX packets 590 bytes 55776 (54.4 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 296 bytes 31781 (31.0 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Παρατηρούμε ότι η IP του επιτιθέμενου (Kali Linux) είναι η 192.168.1.109 με την εντολή **ifconfig**.

```
kalaitzon@ubuntudesktop1:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens32: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:f5:51:1f brd ff:ff:ff:ff:ff:ff
    altname enp2s0
    inet 172.20.10.4/23 brd 172.20.11.255 scope global noprefixroute ens32
        valid_lft forever preferred_lft forever
    inet 192.168.1.108/24 brd 192.168.1.255 scope global dynamic noprefixroute ens32
        valid_lft 86053sec preferred_lft 86053sec
kalaitzon@ubuntudesktop1:~$ ip route
default via 192.168.1.1 dev ens32 proto dhcp src 192.168.1.108 metric 100
172.20.10.0/23 dev ens32 proto kernel scope link src 172.20.10.4 metric 100
192.168.1.0/24 dev ens32 proto kernel scope link src 192.168.1.108 metric 100
kalaitzon@ubuntudesktop1:~$
```

Παρατηρούμε επίσης στο Ubuntu (θύμα) παρατηρούμε ότι η IP του είναι η 192.168.1.108 με την εντολή **ip a** και ότι η IP του gateway/router είναι η 192.168.1.1 με την εντολή **ip route**.

Τώρα με το εργαλείο nmap θα σαρώσουμε το υποδίκτυο και θα δούμε ποιες IP είναι ενεργές (ανάμεσα στις οποίες είναι και αυτή του θύματος). Αυτό θα γίνει με την εντολή **sudo nmap -sn 192.168.1.0/24**. Ο λόγος που βάζουμε /24 είναι για να «πιάσει» κάθε ενεργή IP της μάσκας του δικτύου (που ξεκινάει από .0 έως .254) και να μας τις εμφανίσει.

```
(kali@kali)-[~]
$ nmap -sn 192.168.1.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-03 10:27 EST
Nmap scan report for 192.168.1.1
Host is up (0.019s latency)
MAC Address: [redacted] (TP-Link Limited) router
Nmap scan report for 192.168.1.103
Host is up (0.020s latency).
MAC Address: [redacted] device
Nmap scan report for 192.168.1.107
Host is up (0.0013s latency).
MAC Address: [redacted] device
Nmap scan report for 192.168.1.108
Host is up (0.0050s latency)
MAC Address: 00:0C:29:F5:51:1F (VMware)
Nmap scan report for 192.168.1.109
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 15.16 seconds
```

Από τη σάρωση (nmap -sn 192.168.1.0/24) παρατηρούμε ότι η διεύθυνση 192.168.1.1 είναι ο gateway/router. Η διεύθυνση 192.168.1.109 αντιστοιχεί στο επιτιθέμενο μηχάνημα (Kali Linux) όπως γνωρίζουμε από το ifconfig, ενώ η 192.168.1.108 είναι του θύματος (Ubuntu) διότι οι υπόλοιπες IP είναι από συσκευές μου. Οπότε με σιγουριά γνωρίζουμε την IP του θύματος. Ωστόσο για επιβεβαίωση μπορούμε να το διασταυρώσουμε και από την MAC Address. Με την εντολή **sudo arp-scan --localnet** μπορούμε να δούμε ποια IP διεύθυνση αντιστοιχεί με τη δική της MAC Address. Μπορούμε να το διασταυρώσουμε από την εντολή **ip a** που χρησιμοποιήσαμε παραπάνω στο Ubuntu.

```
(kali@kali)-[~]
$ sudo arp-scan --localnet
Interface: eth0, type: EN10MB, MAC: 00:0c:29:fe:e8:b5, IPv4: 192.168.1.109
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.1.1 [redacted] (Unknown)
192.168.1.101 [redacted] (Unknown)
192.168.1.103 [redacted] (Unknown)
192.168.1.107 [redacted] (Unknown)
192.168.1.108 [redacted] (Unknown)
192.168.1.100 [redacted] (Unknown: locally administered)
192.168.1.101 [redacted] (Unknown) (DUP: 2)
```


Υπό άλλες συνθήκες αυτό συνεχίζει να μη στέκει διότι ως επιτιθέμενος δε γνωρίζω κάτι σχετικά με το OS. Με την εντολή **nmap -vv -O -sV -Pn 192.168.1.108** (όπου -vv για το verbose, -O για να εντοπισει το OS, -sV για να εντοπίσει κάποιο service και -Pn για να μην κάνει ping τη συγκεκριμένη διεύθυνση εφόσον ξέρουμε ότι είναι ενεργή) κανονικά θα έπαιρνα τη πληροφορία ότι το OS είναι Ubuntu. Αλλά λόγω του ότι είναι το πιο πρόσφατο με τις τελευταίες ενημερώσεις, δεν γίνεται σωστή αναγνώριση από το nmap και δε μας δίνεται αυτή η απαραίτητη πληροφορία όπως φαίνεται στην παρακάτω εικόνα. Στο μόνο που μας επιβεβαιώνει είναι ότι το συγκεκριμένο μηχάνημα με αυτή την IP είναι OS αλλά δε μπορεί να αναγνωρίσει κάτι παραπάνω (Too many fingerprints match this host to give specific OS details). Ωστόσο στην περίπτωση που γνωρίζουμε τις άλλες IP (ότι ανήκουν σε άλλες συσκευές) όπως τώρα είναι αρκετό.

```
(kali@kali)-[~]$ nmap -vv -O -sV -Pn 192.168.1.108
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-03 10:30 EST
NSE: Loaded 47 scripts for scanning.
Initiating ARP Ping Scan at 10:30
Scanning 192.168.1.108 [1 port]
Completed ARP Ping Scan at 10:30, 0.09s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host, at 10:30
Completed Parallel DNS resolution of 1 host, at 10:31, 6.62s elapsed
Initiating SYN Stealth Scan at 10:31
Scanning 192.168.1.108 [1000 ports]
Completed SYN Stealth Scan at 10:31, 0.27s elapsed (1000 total ports)
Initiating Service scan at 10:31
Initiating OS detection (try #1) against 192.168.1.108
Retrying OS detection (try #2) against 192.168.1.108
NSE: Script scanning 192.168.1.108.
NSE: Starting runlevel 1 (of 2) scan.
Initiating NSE at 10:31
Completed NSE at 10:31, 0.00s elapsed
NSE: Starting runlevel 2 (of 2) scan.
Initiating NSE at 10:31
Completed NSE at 10:31, 0.00s elapsed
Nmap scan report for 192.168.1.108
Host is up, received arp-response (0.0020s latency).
Scanned at 2025-11-03 10:31:03 EST for 1s
All 1000 scanned ports on 192.168.1.108 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 00:0C:29:F5:51:1F (VMware)
Too many fingerprints match this host to give specific OS details
TCP/IP fingerprint:
SCAN(V=7.95NE=4ND=11/3%OT=NCT=1%CU=43309%PV=Y%DS=1%DC=DNM=000C29%TM=6908CAB9%P=x86_64-pc-linux-gnu)
SEQ(CI=2%II=1)
TS(R=Y%DF=Y%T=40%W=0%S=Z%A=5+NF=AR%O=%RD=0%Q=)
T6(R=Y%DF=Y%T=40%W=0%S=Z%A=5+NF=AR%O=%RD=0%Q=)
T7(R=Y%DF=Y%T=40%W=0%S=Z%A=5+NF=AR%O=%RD=0%Q=)
U1(R=Y%DF=N%T=40%IPL=164%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK=G%RUD=G)
IE(R=Y%DFI=N%T=40%CD=5)

Network Distance: 1 hop

Read data files from: /usr/share/nmap
```

Στη συνέχεια κάναμε ping σε κάθε διεύθυνση IP για να επαληθεύσουμε τη βασική συνδεσιμότητα (reachability) πριν προχωρήσουμε σε arpspoofing από τα Kali Linux. Με αυτό επιβεβαιώνουμε ότι οι IP διευθύνσεις είναι up και ότι υπάρχει δρομολόγηση προς την κάθε IP.

```

(kali㉿kali)-[~]
$ ping 192.168.1.108
PING 192.168.1.108 (192.168.1.108) 56(84) bytes of data.
64 bytes from 192.168.1.108: icmp_seq=1 ttl=64 time=6.91 ms
64 bytes from 192.168.1.108: icmp_seq=2 ttl=64 time=1.93 ms
64 bytes from 192.168.1.108: icmp_seq=3 ttl=64 time=1.72 ms
64 bytes from 192.168.1.108: icmp_seq=4 ttl=64 time=1.61 ms
^C
— 192.168.1.108 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 1.608/3.040/6.910/2.236 ms

(kali㉿kali)-[~]
$ ping 192.168.1.109
PING 192.168.1.109 (192.168.1.109) 56(84) bytes of data.
64 bytes from 192.168.1.109: icmp_seq=1 ttl=64 time=0.126 ms
64 bytes from 192.168.1.109: icmp_seq=2 ttl=64 time=0.131 ms
64 bytes from 192.168.1.109: icmp_seq=3 ttl=64 time=0.123 ms
^C
— 192.168.1.109 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2039ms
rtt min/avg/max/mdev = 0.123/0.126/0.131/0.003 ms

(kali㉿kali)-[~]
$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=22.1 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=11.7 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=8.37 ms
^C
— 192.168.1.1 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 8.368/14.029/22.052/5.830 ms

```

Το ίδιο κάναμε και από τα Ubuntu όπως φαίνεται στην παρακάτω εικόνα.

```

kalaitron@ubuntudesktop1:~$ ping 192.168.1.109
PING 192.168.1.109 (192.168.1.109) 56(84) bytes of data.
64 bytes from 192.168.1.109: icmp_seq=1 ttl=64 time=1.24 ms
64 bytes from 192.168.1.109: icmp_seq=2 ttl=64 time=4.38 ms
64 bytes from 192.168.1.109: icmp_seq=3 ttl=64 time=1.96 ms
^C
--- 192.168.1.109 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 1.242/2.525/4.379/1.342 ms
kalaitron@ubuntudesktop1:~$ ping 192.168.1.108
PING 192.168.1.108 (192.168.1.108) 56(84) bytes of data.
64 bytes from 192.168.1.108: icmp_seq=1 ttl=64 time=0.068 ms
64 bytes from 192.168.1.108: icmp_seq=2 ttl=64 time=0.108 ms
64 bytes from 192.168.1.108: icmp_seq=3 ttl=64 time=0.110 ms
^C
--- 192.168.1.108 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2069ms
rtt min/avg/max/mdev = 0.068/0.095/0.110/0.019 ms
kalaitron@ubuntudesktop1:~$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1) 56(84) bytes of data.
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=91.3 ms
64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=7.74 ms
64 bytes from 192.168.1.1: icmp_seq=3 ttl=64 time=8.61 ms
^C
--- 192.168.1.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2006ms
rtt min/avg/max/mdev = 7.738/35.890/91.327/39.201 ms
kalaitron@ubuntudesktop1:~$

```

```

(kali@kali)-[~]
$ sudo ip link set dev eth0 promisc on
[sudo] password for kali:
Sorry, try again.
[sudo] password for kali:

(kali@kali)-[~]
$ ip link show eth0
2: eth0: <BROADCAST,MULTICAST,PROMISC,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:fe:e8:b5 brd ff:ff:ff:ff:ff:ff

```

Ενεργοποιούμε το promiscuous mode με την εντολή **sudo ip link set dev eth0 promisc on** και το ελέγχουμε με την εντολή **ip link show eth0** για να λαμβάνουμε όλη την κίνηση του interface και να αναλύσουμε πακέτα.

```

(kali@kali)-[~]
$ echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
1

(kali@kali)-[~]
$ cat /proc/sys/net/ipv4/ip_forward
1

```

Στη συνέχεια θα γράψουμε την τιμή 1 στο αρχείο συστήματος `/proc/sys/net/ipv4/ip_forward` (με την εντολή **`echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward`**). Σε MITM και arpspoof σενάριο, αν ο επιτιθέμενος (Kali Linux) «παίρνει» την κίνηση ανάμεσα σε θύμα και gateway, πρέπει να διαβιβάζει τα πακέτα προς το πραγματικό gateway ώστε το θύμα να συνεχίσει να έχει Internet. Χωρίς IP forwarding, ο επιτιθέμενος θα δεχόταν τα πακέτα αλλά δεν θα τα προωθούσε, με αποτέλεσμα το θύμα να χάσει σύνδεση. και στη συνέχεια ελέγξαμε την τιμή με `cat /proc/sys/net/ipv4/ip_forward`, που επέστρεψε 1, πράγμα που σημαίνει ότι η παράμετρος `net.ipv4.ip_forward` είναι ενεργοποιημένη. Μετά θα ελέγξουμε τη συγκεκριμένη τιμή με την εντολή **`cat /proc/sys/net/ipv4/ip_forward`**. Εφόσον επιστρέφει την τιμή 1, έχει ενεργοποιηθεί.

Το παραπάνω αφορά μόνο το IPv4 γιατί το arpspoof λειτουργεί μέσω του πρωτοκόλλου ARP (Address Resolution Protocol) και επηρεάζει αποκλειστικά IPv4 κυκλοφορία. Το ARP είναι ένα πρωτόκολλο που μεταφράζει μια διεύθυνση IPv4 σε φυσική διεύθυνση δικτύου. Χρησιμοποιείται ώστε ένα μηχάνημα να ξέρει σε ποια MAC Address να στείλει το πακέτο όταν θέλει να επικοινωνήσει με μια IP στο ίδιο τοπικό δίκτυο (LAN). Το IPv6 δεν χρησιμοποιεί ARP αλλά το πρωτόκολλο NDP (Neighbor Discovery Protocol). Οπότε τα πακέτα IPv6 δεν επηρεάζονται από arpspoof. Στην πράξη αυτό σημαίνει ότι σε περιβάλλον dual-stack (IPv4+IPv6) οι σύγχρονοι browsers και υπηρεσίες τείνουν να δοκιμάζουν IPv6 πρώτα και αν υπάρχει έγκυρη IPv6 διαδρομή, η κίνηση «ξεφεύγει» από το arpspoof. Για αυτό το λόγο παρατηρήσαμε καθυστέρηση στο «άνοιγμα» των ιστοσελίδων. Έκανε δηλαδή προσπάθεια μέσω IPv6, καθυστερούσε και επέστρεφε σε μορφή IPv4 ή και το ανάποδο (συνήθως αυτό). Απαραίτητη όμως προϋπόθεση, την οποία δε γνωρίζαμε μέχρι να αντιληφθούμε τι συμβαίνει, ήταν να είναι οπωσδήποτε ενεργό το IPv6 και όχι απενεργοποιημένο όπως το είχαμε στην αρχή. Με απενεργοποιημένο το IPv6 φόρτωναν οι ιστοσελίδες και οι υπηρεσίες και στο τέλος εμφάνιζαν το μήνυμα «Εκτός σύνδεσης». Αυτό γινόταν διότι όπως προαναφέραμε, οι περισσότερες ιστοσελίδες και υπηρεσίες υποστηρίζουν IPv6 (ή τουλάχιστον αυτές που δοκίμασαμε) και δε μπορούσαν να φορτώσουν μέσω IPv4 που ήταν το μόνο ενεργοποιημένο πρωτόκολλο. Μόλις ενεργοποιήσαμε το IPv6 φόρτωσαν οι ιστοσελίδες κανονικά.

Τίσως με ένα άλλο εργαλείο όπως το bettercap που υποστηρίζει ARP και NDP πρωτόκολλα ή το mitm6, ειδικά για IPv6/DNS takeover, να γινόταν πιο εύκολα το MITM σενάριο.

[illegible]

Τώρα θα χρησιμοποιήσουμε το εργαλείο `arp spoof` για να ξεκινήσει η MITM παρακολούθηση της επικοινωνίας του θύματος (Ubuntu) και `gateway`. Συγκεκριμένα με την εντολή **`sudo arp spoof -i eth0 -t 192.168.1.108 192.168.1.1`** όπου πρακτικά λέμε στο θύμα (Ubuntu) ότι εγώ (Kali Linux) είμαι το `gateway`. Με άλλα λόγια, η εντολή στέλνει ARP replies στον στόχο 192.168.1.108, λέγοντάς του ψευδώς ότι η IP 192.168.1.1 (το `gateway`) έχει τη MAC Address του επιτιθέμενου (Kali Linux).

[illegible]

Συνεχίζουμε σε ξεχωριστό terminal με την εντολή **sudo arpspoof -i eth0 -t 192.168.1.1 192.168.1.108** όπου πρακτικά λέμε στο gateway ότι εγώ είμαι το θύμα (Ubuntu). Με άλλα λόγια αντίστοιχα, η εντολή λέει στο gateway ότι η IP του θύματος (192.168.1.108) έχει τη MAC του επιτιθέμενου (Kali Linux). Έτσι, και τα δύο άκρα (θύμα και gateway) στέλνουν τα πακέτα τους στον επιτιθέμενο και ως αποτέλεσμα έχουμε MITM.

```
(kali@kali)-[~]
$ ps aux | grep arpspoof
root      15278  0.0  0.1  22380  8296 pts/0    S+   10:35   0:00 sudo arpspoof -i eth0 -t 192.168.1.108 192.168.1.1
root      15280  0.0  0.0  22380  2748 pts/1    Ss   10:35   0:00 sudo arpspoof -i eth0 -t 192.168.1.108 192.168.1.1
root      15281  0.0  0.0    5468  2716 pts/1    S+   10:35   0:00 arpspoof -i eth0 -t 192.168.1.108 192.168.1.1
root      15604  0.0  0.1  22388  8288 pts/2    S+   10:35   0:00 sudo arpspoof -i eth0 -t 192.168.1.1 192.168.1.108
root      15624  0.0  0.0  22388  2864 pts/3    Ss   10:36   0:00 sudo arpspoof -i eth0 -t 192.168.1.1 192.168.1.108
root      15625  0.0  0.0    5468  2712 pts/3    S+   10:36   0:01 arpspoof -i eth0 -t 192.168.1.1 192.168.1.108
kali      32655  0.0  0.0    6544  2412 pts/6    S+   11:10   0:00 grep --color=auto arpspoof

(kali@kali)-[~]
$ sudo pkill arpspoof
[sudo] password for kali:

(kali@kali)-[~]
$
```

Χρήσιμες εντολές για το arpspoof:

sudo pkill arpspoof | κλείνει όλες τις διεργασίες arpspoof

ps aux | grep arpspoof | ελέγχει αν υπάρχουν ακόμα διεργασίες με αυτό το όνομα

Στη συνέχεια κάνουμε compile το packet sniffer που βρίσκεται στον φάκελο Downloads. Αφού ολοκληρωθεί η μεταγλώττιση (**gcc pcap_sniffer.c -o pcap_sniffer -lpcap**), θα τρέξουμε το sniffer στο interface eth0 όπως κάναμε στο Case 2. Θα ξεκινήσει η καταγραφή των πακέτων και θα σταματήσει μόλις επιλέξουμε τα κουμπιά Ctrl + C στο πληκτρολόγιο. Θα αποθηκευτούν τα πακέτα στο .pcap αρχείο το οποίο θα ανοίξουμε μέσω του wireshark για την εξαγωγή των αποτελεσμάτων.

```

zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ cd Downloads

(kali@kali)-[~/Downloads]
$ ls -l *.c
-rw-rw-r-- 1 kali kali 2554 Oct 31 11:41 pcap_sniffer.c

(kali@kali)-[~/Downloads]
$ gcc pcap_sniffer.c -o pcap_sniffer -lpcap

(kali@kali)-[~/Downloads]
$ sudo ./pcap_sniffer eth0
[sudo] password for kali:
Capturing packets on eth0... Press Ctrl+C to stop.
Captured packet (42 bytes)
Captured packet (60 bytes)
Captured packet (42 bytes)
Captured packet (42 bytes)
Captured packet (82 bytes)
Captured packet (412 bytes)
Captured packet (102 bytes)
Captured packet (444 bytes)
Captured packet (60 bytes)
Captured packet (42 bytes)
Captured packet (42 bytes)
Captured packet (118 bytes)
Captured packet (110 bytes)
Captured packet (110 bytes)
Captured packet (60 bytes)
Captured packet (446 bytes)
Captured packet (466 bytes)
Captured packet (42 bytes)
Captured packet (42 bytes)
Captured packet (60 bytes)
Captured packet (42 bytes)

```

```

(kali@kali)-[~/Downloads]
$ wireshark capture_libpcap.pcap
** (wireshark:39307) 11:24:34.647491 [GUI ECHO] -- virtual const QPalette* Qt6CTPlatformTheme::palette(QPlatformTheme::Palette) const QPlatformTheme::SystemPalette
** (wireshark:39307) 11:24:34.652403 [GUI ECHO] -- virtual const QPalette* Qt6CTPlatformTheme::palette(QPlatformTheme::Palette) const QPlatformTheme::ToolButtonPalette
** (wireshark:39307) 11:24:34.652673 [GUI ECHO] -- virtual const QPalette* Qt6CTPlatformTheme::palette(QPlatformTheme::Palette) const QPlatformTheme::ToggleButtonPalette
** (wireshark:39307) 11:24:34.652783 [GUI ECHO] -- virtual const QPalette* Qt6CTPlatformTheme::palette(QPlatformTheme::Palette) const QPlatformTheme::CheckBoxPalette
** (wireshark:39307) 11:24:34.652818 [GUI ECHO] -- virtual const QPalette* Qt6CTPlatformTheme::palette(QPlatformTheme::Palette) const QPlatformTheme::RadioButtonPalette

```

Ανοίγοντας το αρχείο `capture_libpcap.pcap` με την εντολή (**wireshark** `capture_libpcap.pcap`) με το wireshark εντοπίσαμε και καταγράψαμε την κυκλοφορία πακέτων μεταξύ του θύματος (Ubuntu) και του gateway. Στο capture εμφανίζονται πακέτα και αιτήματα προς διάφορες ιστοσελίδες και υπηρεσίες που επισκεφθήκαμε κατά τη διάρκεια της δοκιμής (π.χ. Wikipedia, YouTube, Reddit), επιβεβαιώνοντας την επιτυχή καταγραφή της δικτυακής δραστηριότητας.

Συμπέρασμα:

Μέσω της άσκησης αποδείχθηκε στην πράξη πώς μπορεί να πραγματοποιηθεί επίθεση τύπου Man in the Middle (MITM) με τη χρήση του εργαλείου arpspoof, το οποίο παραπλανεί το θύμα και το getaway ώστε η επικοινωνία τους να περνά μέσω του επιτιθέμενου. Με την ενεργοποίηση του IP forwarding και την ταυτόχρονη καταγραφή της δικτυακής κίνησης με το Wireshark, κατέστη δυνατή η παρακολούθηση των πακέτων που ανταλλάσσονται ανάμεσά τους. Συνολικά, η άσκηση ανέδειξε τη λειτουργία του arpspoof εργαλείου και επιβεβαίωσε τη δυνατότητα υποκλοπής και παρακολούθησης πακέτων σε επίπεδο τοπικού δικτύου (LAN).

Βιβλιογραφία

danscourses. (2012, March 27). *ARP Spoofing - Man-in-the-middle attack* [Video file]. Retrieved from https://www.youtube.com/watch?v=hI9J_tnNDCc

Narten, T., Nordmark, E., & Simpson, W. (1998). *Neighbor Discovery for IP Version 6 (IPv6)*. <https://doi.org/10.17487/rfc2461>

Introduction. (n.d.). Bettercap. <https://www.bettercap.org/project/introduction/>

Tools of the Trade: IPv6 DNS Takeover with MitM6. (2023, September 22). <https://www.evolvesecurity.com/blog-posts/tools-of-the-trade-ipv6-dns-takeover-with-mitm6>

Kampourakis, V., Kambourakis, G., Chatzoglou, E., & Zaroliagis, C. (2022). Revisiting man-in-the-middle attacks against HTTPS. *Network Security*, 2022(3). [https://doi.org/10.12968/s1353-4858\(22\)70028-1](https://doi.org/10.12968/s1353-4858(22)70028-1)

Vaishnavi. (2025, June 19). How to Use Bettercap for Network Penetration Testing ? Beginner's Guide with Commands and Use Cases. *WebAsha Technologies*. <https://www.webasha.com/blog/how-to-use-bettercap-for-network-penetration-testing-beginners-guide-with-commands-and-use-cases>

GeeksforGeeks. (2025, October 10). *Address Resolution Protocol ARP*. GeeksforGeeks. <https://www.geeksforgeeks.org/computer-networks/arp-protocol/>

ChatGPT. (n.d.). Retrieved from <https://chatgpt.com/>

Kumar, G. (2025, July 15). *IPv6 Neighbor Discovery Protocol (NDP) explained*. <https://www.uninets.com/UniNets.https://www.uninets.com/blog/neighbor-discovery-protocol>

Infosec. (2021, April 27). *How to set up a man in the middle attack | Free Cyber Work Applied series* [Video]. YouTube. <https://www.youtube.com/watch?v=GkexkyUbUd4>

Tech Sky - Ethical Hacking. (2024, September 15). *How to Spy on Any Device's Network using ARP Spoofing in Kali Linux?* [Video]. YouTube. <https://www.youtube.com/watch?v=YtLsI-wHv2U>

CertBros. (2021, April 6). *ARP Poisoning | Man-in-the-Middle attack* [Video]. YouTube. <https://www.youtube.com/watch?v=A7nih6SANYs>

Chris Greer. (2021, December 9). *How ARP poisoning works // Man-in-the-Middle* [Video]. YouTube. <https://www.youtube.com/watch?v=cVTUeEoJgEg>